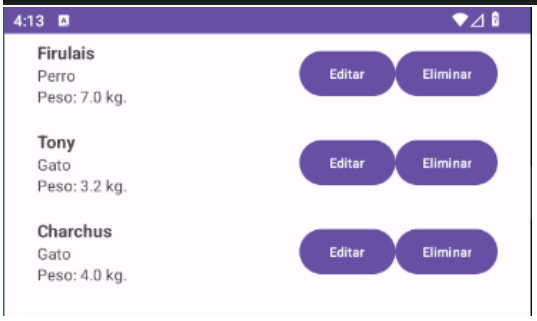
Documentación del proceso: “Editar Mascota”

Paso N.-01. Activar el editar desde el Recyclerview

Cuando el usuario pulsa el botón “editar” de un item, el adapter notifica al ListarCustom mediante la interfaz OnAccionListener. El método onEditar recibe la posición y el objeto mascota con todos sus datos disponilbles y llama a editarMascota.

```
@Override
public void onEditar(int position, Mascota mascota) {
    editarMascota(mascota);
}
```



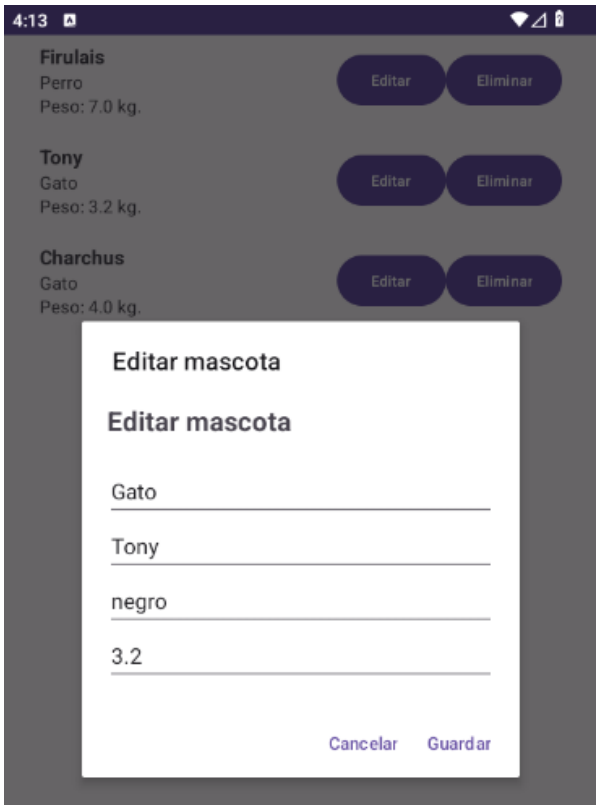
2. Inflar el formulario y precargar los datos

Se infla el layout activity_editar.xml como vista del AlertDialog. Luego se obtiene la referencia de cada campo y se rellena automáticamente con los datos actuales del objeto mascota, que ya fueron cargados cuando se listaron las mascotas, por lo que no se necesita una consulta extra al servidor…

```
private void editarMascota(Mascota mascota) {
    View viewDialog = getLayoutInflater()
            .inflate(R.layout.activity_editar, root: null);

    EditText edtTipo   = viewDialog.findViewById(R.id.edtTipo);
    EditText edtNombre = viewDialog.findViewById(R.id.edtNombre);
    EditText edtColor  = viewDialog.findViewById(R.id.edtColor);
    EditText edtPeso   = viewDialog.findViewById(R.id.edtPeso);

    edtTipo.setText(mascota.getTipo());
    edtNombre.setText(mascota.getNombre());
    edtColor.setText(mascota.getColor());
    edtPeso.setText(String.valueOf(mascota.getPesokg()));
```



3. Construir el JSON y enviar el “PUT”

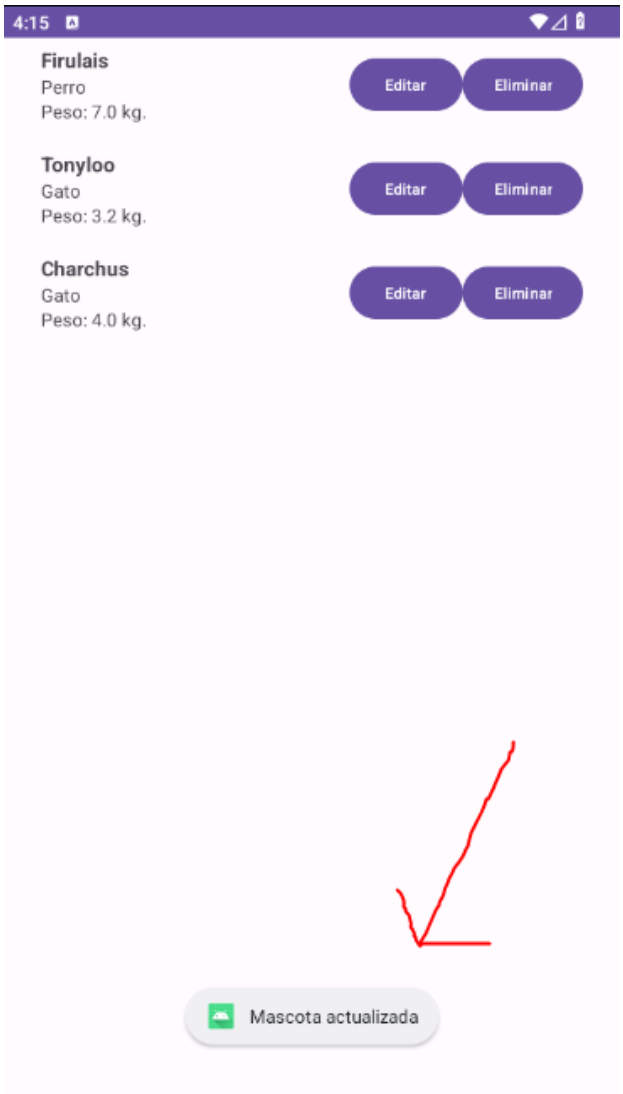
Al pulsar ‘Guardar’ se construye un JSONObject con los valores modificados por el usuario. Se realiza una petición PUT a /mascotas/:id enviando ese JSON. Si el servidor responde exitosamente y de manera correcta, se debe de mostrar un Toast y se recarga la lista.

```
builder.setPositiveButton( text: "Guardar", ( DialogInterface dialog, int which) -> {
    try {
        JSONObject body = new JSONObject();
        body.put( name: "tipo",   edtTipo.getText().toString());
        body.put( name: "nombre", edtNombre.getText().toString());
        body.put( name: "color",  edtColor.getText().toString());
        body.put( name: "pesokg", Double.parseDouble(edtPeso.getText().toString()));

        String urlEditar = this.URL + mascota.getId();

        JsonObjectRequest putRequest = new JsonObjectRequest(
                Request.Method.PUT,
                urlEditar,
                body,
                JSONObject response -> {
                    Toast.makeText( context: this,  text: "Mascota actualizada", Toast.LENGTH_SHORT).show();
                    obtenerDatos();
                },
                VolleyError error -> {
                    Log.e( tag: "ErrorPUT", error.toString());
                    Toast.makeText( context: this,  text: "Error al actualizar", Toast.LENGTH_SHORT).show();
                }
        );
        requestQueue.add(putRequest);

    } catch (Exception e) {
```



4. Servidor recibe y procesa la petición

En el servidor Node.js, el endpoint PUT recibe el id por la url y los nuevos datos por el body. Construye la consulta SQL con comodines para evitar SQL injection y actualiza el registro en Mysql.

```
//Se enviará el ID por la URL (endpoint)
//SE enviarán los daots por JSON
app.put('/mascotas/:id',(req, res) => {
    const { id } = req.params
    const { tipo, nombre, color, pesokg } = req.body

    //Podemos escribir multilineas utilizando
    //Comodines van en un indice
    const sql = `
    UPDATE mascotas SET
    tipo = ?, nombre = ?, color = ?, pesokg = ?
    WHERE id = ?`

    db.query(sql, [tipo, nombre, color, pesokg, id], (err, results) => {
        if (err) {
            res.status(500).send({
                success: false,
                message: 'No se concretó la actualización'
            })
        }

        return res.send({
            success: true,
            message: 'Registro actualizado'
        })
    })
})
```

```
PS C:\Users\PC\OneDrive\Documentos\WS_Mascotas> node index
Servidor iniciado correctamente en http://localhost:3000
Conectado a la BD de mascotas...
```